V-OS

Un sistema operativo grafico per l'Amstrad CPC 6128

Manuale

Generato da docs/manual/it/*.md e sdk/README.it.md con tools/mkdocs.py (make docs).

Indice

V-OS è un sistema operativo grafico per l'Amstrad CPC 6128, scritto da zero in assembly Z80. Offre:

- una barra dei menu, finestre trascinabili, dialoghi e pulsanti, che si usano con la tastiera o con un mouse AMX
- tre viste per file e programmi (Scrivania, Launcher, Commander) e le impostazioni (Impostazioni)
- sei lingue come plugin: inglese, greco, francese, spagnolo, tedesco e italiano
- programmi che girano uno accanto all'altro e riprendono da dove si erano fermati, un accessorio di scrivania e programmi in background
- Studio, un editor a blocchi in stile Scratch che gira sul CPC stesso
- un'API con 42 chiamate e un SDK per programmi in assembly

Questo manuale esiste anche in greco (../00-index.md), inglese (../en/00-index.md), francese (../fr/00-index.md), spagnolo (../es/00-index.md) e tedesco (../de/00-index.md).

- 1. Per iniziare
- 2. Le viste
- 3. I programmi
- 4. Le lingue
- 5. Studio: programmi con i blocchi
- 6. I blocchi sul PC: l'editor web e `vbc.py`
- 7. L'SDK di V-OS: programmi in assembly
- 8. Sviluppare V-OS

1. Per iniziare

Cosa serve

- Un Amstrad CPC 6128 o un suo emulatore. V-OS è testato su Caprice32.
- Il dischetto di V-OS nell'unità A:, cioè il `build/vos.dsk` creato da `make`.
- Facoltativamente:
  - un mouse AMX
  - una seconda unità (B:)
  - un'espansione di memoria (ad es. 512K), per più programmi insieme

Avvio

Dal BASIC:

```
RUN"VOS
```

V-OS carica il kernel, disattiva le ROM e prende il controllo di tutta la macchina. Rileva da solo l'espansione di memoria.

- **Prima vista:** la Scrivania. Si può cambiare nelle Impostazioni, alla riga VISTA INIZIALE.
- **Lingua:** inglese. Se sul dischetto c'è un `SETUP.CFG`, parte con la lingua e le impostazioni salvate lì.
- **Uscita:** FILE > ESCI IN BASIC (CONTROL+Q nelle viste). Chiede conferma e riavvia il CPC.

Lo schermo

V-OS usa tutto lo schermo (overscan): 384×272 pixel in modo 1 (4 colori) o 192×272 in modo 0 (16 colori).

- **Barra dei menu** (in alto): i menu della vista o del programma, di solito FILE o il menu del programma, VISTA e AIUTO. A destra c'è la lingua della tastiera (EN, ΕΛ, ...).
- **Riga in basso:** in ogni vista, i tasti che funzionano lì.

Tasti che funzionano ovunque

Tasto	Cosa fa
F1	Scrivania
F2	Launcher
F3	Commander
F4	Impostazioni
F5	apre e chiude l'accessorio di scrivania (ACCESSORIO)
F6	i programmi in esecuzione, per sceglierne uno
ESC	apre il primo menu. Nei menu, ←/→ cambiano menu, ↑/↓ scelgono una voce, RETURN la esegue, ESC chiude
CONTROL+lettera	le scorciatoie dei menu (ad es. ^Q, ^L)
CONTROL+L	cambia lingua (VISTA > CAMBIA LINGUA)

CONTROL+K	il dialogo con i tasti (AIUTO > TASTI)
CONTROL+SHIFT	passa la tastiera dall'inglese alla lingua del pacchetto e viceversa
TAB, SHIFT+TAB	spostano il focus tra i pulsanti. RETURN, ENTER o SPAZIO premono il pulsante che ha il focus

F1–F6 sono i tasti f1–f6 del tastierino numerico del CPC.

Il puntatore e il mouse

- **Con un mouse AMX:** il puntatore si muove con il mouse, e il suo pulsante fa clic. Il mouse si attiva e si configura nelle Impostazioni (MOUSE AMX, LETTURA AMX).
- **Senza mouse:** il puntatore si muove con i tasti freccia, e COPY fa clic. Nelle viste che usano le frecce per scegliere (Scrivania, Launcher, Commander), le frecce scelgono file e programmi invece di muovere il puntatore.

Quasi tutto funziona in entrambi i modi: un clic su un file o un programma lo seleziona, e un secondo clic lo apre.

Finestre e dialoghi

- **Trascinamento:** ogni finestra si può trascinare per la sua barra del titolo: i dialoghi, la finestra FILE della Scrivania, l'accessorio di scrivania e le finestre dei programmi a blocchi. Mentre tieni premuto il pulsante si vede un contorno tratteggiato, e la finestra viene ridisegnata dove lo rilasci.
- **Dialoghi:** aspettano una risposta. TAB e ←/→ cambiano pulsante, RETURN lo preme, ed ESC sceglie l'ultimo (di solito NO o ANNULLA).
- **Inserimento di testo** (ad es. per rinominare): scrivi alla fine, DEL cancella, RETURN accetta ed ESC annulla. Puoi scrivere anche nella lingua del pacchetto.

Dischetti e file

- **Formato:** V-OS legge e scrive dischetti AMSDOS (formato DATA o SYSTEM) in A: e B:. Anche il BASIC può leggere i suoi file.
- **Nomi:** 8 caratteri più 3 per il tipo (`NAME.TYP`).

Tipi che incontrerai:

Tipo	Cos'è
`.BIN`	un programma V-OS (o un file binario)
`.VBN`	un programma V-OS fatto con i blocchi
`.VBS`	un programma di Studio (il suo sorgente)
`.OVL`, `.MSG`, `.LNG`	parti di V-OS stesso: viste, testi, lingue
`.SYS`	file di sistema (ad es. `VBRUN.SYS`, il player di Studio)
`SETUP.CFG`	le impostazioni

Le impostazioni, le lingue e le viste vengono lette da A:, quindi il dischetto di V-OS deve restare lì.

Messaggi di errore

Gli errori del disco compaiono in un dialogo: UNITÀ NON PRONTA, DISCO PROTETTO, FILE NON TROVATO, DISCO PIENO, DIRECTORY PIENA, NOME GIÀ USATO, NOME NON VALIDO e altri.

Altri due riguardano i programmi:

- **NON È UN PROGRAMMA V-OS:** il file non ha l'intestazione di un programma.
- **CHIUDI PRIMA UN ALTRO PROGRAMMA:** non c'è spazio. Vedi il capitolo 3.

2. Le viste

V-OS ha quattro "viste": modi di vedere i file e i programmi, e le impostazioni. Si cambia vista con F1–F4 o dal menu VISTA. Ogni vista ricorda quale file era selezionato, e la successiva seleziona lo stesso file.

I menu comuni

Gli ultimi due menu sono uguali in tutte le viste e nei programmi:

VISTA		**AIUTO**	
SCRIVANIA	F1	TASTI	^K
PROGRAMMI	F2	INFORMAZIONI SU V-OS	
COMMANDER	F3		
IMPOSTAZIONI	F4		
PROGRAMMI (in esecuzione)	F6		
CAMBIA LINGUA	^L		

AIUTO > INFORMAZIONI SU V-OS mostra la versione (0.1 beta), la data della compilazione, l'autore (Revive8bit, 2026) e l'architetto (Vasper).

Scrivania, F1

Modo 1. Una finestra FILE mostra i file di un'unità come icone, quattro per riga. Il tipo è scritto dentro l'icona e il nome sotto, e un `.VBN` ha accanto un segno di "play". A destra, due icone scelgono A: o B:. La barra del titolo mostra il formato del disco e il numero di file, e lo spazio libero è mostrato in basso.

Tasto	**Cosa fa**
frecce	scelgono un file (↑/↓ una riga)
TAB	l'altra unità
RETURN, ENTER	apri: un programma si avvia, un file di un tipo noto si apre nel suo programma, gli altri mostrano le loro informazioni
^C	copia sull'altra unità
^D	elimina (chiede prima)
^N	rinomina
^R	rilegge i dischi
mouse	clic su un file: selezione, di nuovo: apertura. Clic su un'unità: la apre

- **Menu FILE:** APRI, INFO SUL FILE (^I), COPIA SULL'ALTRA UNITÀ, ELIMINA, RINOMINA, RILEGGI I DISCHI, ESCI IN BASIC.
- **La finestra FILE** si può trascinare per il suo titolo e resta lì, anche quando cambi vista.
- **Apertura per tipo** (anche nel Commander): i `.TXT` si aprono in NOTEPAD, i `.MD` in MARKDOWN, i `.PIC` e `.LYR` in PAINT, i `.MUS` in MUSIC, che comincia a suonarli. Il programma si cerca sull'unità del file, poi su A:, poi su B:. Se non si trova, appare "NESSUN PROGRAMMA PER QUESTO FILE". Se il programma ha modifiche non salvate, chiede prima.

Launcher (PROGRAMMI), F2

Modo 0, 16 colori. Mostra solo i programmi di un'unità (`.BIN`, `.VBN`, `.BAS`), all'inizio A:, come

riquadri in una griglia 3×3, con le pagine successive sotto. La riga della pagina comincia con l'unità (`A:`, `B:`). Ogni programma che ha un'icona la mostra sul suo riquadro. Il riquadro selezionato è bianco.

Tasto	Cosa fa
frecce	scelgono (↑/↓ una riga. Oltre il bordo, la pagina successiva o precedente)
RETURN, ENTER	avvia il programma. Se è già in esecuzione, riprende da dove era
^A	lo carica come accessorio di scrivania (COME ACCESSORIO). Poi F5 lo apre
^R	rilegge il dischetto
TAB	l'altra unità (ALTRA UNITÀ). Un programma avviato da B: torna a B:
mouse	clic: selezione, di nuovo: avvio

Menu FILE: AVVIA, COME ACCESSORIO, ALTRA UNITÀ, RILEGGI I DISCHI, ESCI IN BASIC.

Commander, F3

Modo 1, due pannelli con elenchi di file: A: a sinistra, B: a destra. Ogni pannello mostra nel titolo il formato e il numero di file, e in basso lo spazio libero. Il pannello attivo ha il titolo scuro e una selezione.

Tasto	Cosa fa
↑/↓	scelgono (con SHIFT: una pagina)
TAB, ←/→	l'altro pannello
RETURN, ENTER	apri, come nella Scrivania. Le informazioni (dimensione, tipo, indirizzo di caricamento, i primi byte) con ^I
^C	copia nell'altro pannello, di qualsiasi dimensione. Se c'è un file con lo stesso nome, chiede prima
^D	elimina
^N	rinomina
^R	rilegge entrambi i dischi
mouse	clic: selezione, di nuovo: apertura

Menu FILE: APRI, INFO SUL FILE (^I), COPIA DI FRONTE, ELIMINA, RINOMINA, RILEGGI I DISCHI, FORMATTA IL DISCO (DATA), ESCI IN BASIC.

> FORMATTA IL DISCO cancella tutti i file sul dischetto del pannello attivo. Chiede prima.

Impostazioni, F4

Modo 1. Una riga per ogni impostazione. Le modifiche hanno effetto subito. Quando vengono salvate (RETURN o ^S) sono scritte in `A:SETUP.CFG` e valgono anche al prossimo avvio.

Impostazione	Valori
VISTA INIZIALE	SCRIVANIA, PROGRAMMI, COMMANDER
LINGUA	ENGLISH, ΕΛΛΗΝΙΚΑ, FRANÇAIS, ESPAÑOL, DEUTSCH, ITALIANO
RITARDO RIPETIZ.	quanto a lungo tieni premuto un tasto prima che inizi a ripetersi: BREVE, MEDIO, LUNGO
VELOC. RIPETIZ.	LENTA, MEDIO, VELOCE
MOUSE AMX	SÌ, NO
LETTURA AMX	IMPULSI o LIVELLI: come viene letto il mouse. Se il puntatore non si muove bene, prova l'altro

COLORE SFONDO	BLU, NERO, VERDE, ROSSO

Tasto	Cosa fa
↑/↓	scelgono un'impostazione
←/→	cambiano il valore (anche un clic sulla riga)
RETURN, ^S	salvano

Cambiare la LINGUA carica subito il pacchetto di lingua da A:. Vedi il capitolo 4.

3. I programmi

Avvio e chiusura

Un programma si avvia in uno di questi modi:

- dal Launcher: selezionalo e premi RETURN, oppure fai clic due volte
- dalla Scrivania o dal Commander: FILE > AVVIA
- da un altro programma, come Studio, che avvia quello che crei

Ogni programma ha il proprio menu accanto ai comuni VISTA e AIUTO. Si chiude dal suo menu (di solito BACK o STOP, CONTROL+Q) e torna alla vista da cui era stato avviato.

Più programmi insieme

I programmi V-OS sono di due tipi:

- **Piccoli, con una tabella di rilocazione:** DEMO, MIXED e CLOCK, gli esempi dell'editor web (PONG, CATCH, STARCAT, COUNTER) e tutto quello che crea l'SDK. Condividono 12K di memoria, e ne girano insieme quanti ce ne stanno.
- **Programmi che hanno bisogno di tutta la memoria:** STUDIO e quello che crea Studio (CALC, MINES, FLAP, TICKER e i tuoi). Senza un'espansione di memoria girano da soli. Se uno è già in esecuzione, compare "CHIUDI PRIMA UN ALTRO PROGRAMMA".

Quando passi a una vista (F1–F4) mentre un programma è in esecuzione:

- Un programma piccolo resta in memoria. Lo riprendi dal Launcher (RETURN su di esso) o con F6, e riprende esattamente da dove era.
- Un programma che ha tutta la memoria si chiude, a meno che non ci sia un'espansione di memoria.

F6 (VISTA > PROGRAMMI) mostra l'elenco dei programmi in esecuzione. Ne scegli uno con le frecce e RETURN. Se non ce n'è nessuno, compare "NESSUN PROGRAMMA IN ESECUZIONE".

Con un'espansione di memoria (ad es. 512K, che V-OS trova da solo all'avvio), ogni programma che ha bisogno di tutta la memoria riceve un proprio banco. Così STUDIO, CALC, DEMO e PONG possono girare tutti insieme.

In background

Un programma dietro un altro, o dietro una vista, normalmente **si blocca** e riprende quando torna in primo piano.

Alcuni programmi chiedono di **continuare a girare** anche in background, ad es. TICKER, che conta il tempo. Mentre sono lì non disegnano e non suonano. Quando tornano in primo piano, hanno contato anche il tempo in cui erano via.

In Studio lo scegli per il tuo programma con STUDIO > IN BACKGROUND.

L'accessorio di scrivania (ACCESSORIO)

Un programma piccolo, come CLOCK, può aprirsi in una finestra **sopra qualsiasi cosa**: una vista o un programma.

1. Nel Launcher, seleziona CLOCK e scegli FILE > COME ACCESSORIO (^A).

2. Da quel momento, **F5** lo apre e lo chiude, ovunque tu sia. Anche ESC lo chiude.

Mentre è aperto, riceve i tasti e i clic. La sua finestra si può trascinare per il titolo. Quando si chiude, quello che c'era sotto viene ridisegnato.

I programmi sul dischetto

Programma	Cos'è
STUDIO	l'editor a blocchi: crei programmi sul CPC (capitolo 5)
DEMO	il primo programma V-OS: un quadrato che si muove da solo. Le frecce lo guidano e SPAZIO ne cambia il colore
MIXED	lo "schermo misto": una fascia a 16 colori (modo 0) tra fasce in modo 1. Un clic mostra le coordinate
CLOCK	il tempo trascorso dall'avvio di V-OS. Gira anche come accessorio di scrivania
CALC	una calcolatrice in una finestra con pulsanti, fatta in Studio (c'è anche `CALC.VBS`)
MINES	un campo minato 8×6 con 8 mine, fatto in Studio. Un clic apre una casella, e NEW inizia una nuova partita
BRICKS	rompi-mattoni: una racchetta con ←/→, 32 mattoni, 3 vite, livelli con una palla sempre più veloce. A GAME OVER, SPAZIO per una nuova partita
VROUVES	tower defence, dal gioco per CPC LET'S GO FOR VROUVES: 7 livelli, dal Kafenio al cimitero, con 10 ondate ciascuno. È sul suo dischetto, `vrouves.dsk`: in B:, e TAB nel Launcher. Le frecce muovono il cursore, 1–6 sceglie un difensore, SPAZIO costruisce, S vende, N chiama l'ondata successiva, P mette in pausa. Punti per ogni invasore e per gli abitanti rimasti alla fine di un livello. Il dischetto conserva `VROUVES.SAV`: CONTINUE riparte dal livello raggiunto, HIGH SCORES mostra i cinque migliori con i nomi. Scritto con l'SDK (`myapps/vrouves`)
NOTEPAD	un editor di testo in modo 1, 48×22, per file fino a 4K. NEW, OPEN (solo file `.TXT`), SAVE, SAVE AS, OTHER DRIVE. Frecce, DEL, CLR e RETURN; un clic posiziona il cursore. La riga in basso mostra riga e colonna. Menu EDIT: COPY ^C, CUT ^X, PASTE ^V, SELECT ALL ^A; SHIFT e le frecce selezionano. Gli appunti (fino a 1K) sono comuni a tutti i programmi (`myapps/notepad`)
PAINT	disegno in modo 0 con 16 colori: un'immagine 128×128 (`.PIC`), o 64×64 con zoom 2× in 3 livelli (`.LYR`, NEW LAYERS). Strumenti: P matita, B pennello, L linea, X rettangolo, F riempimento; TAB cambia il colore. Frecce e COPY disegnano senza mouse. Livelli: 1–3 ne sceglie uno, H lo nasconde o lo mostra, U e D lo spostano su e giù, E è la gomma (`myapps/paint`)
MUSIC	suona brani di Arkos Tracker 3 (`.MUS`, fatti con `make songs`), con tre barre del volume. È sul suo dischetto, `music.dsk`, in B:. OPEN, SPAZIO suona e ferma. La musica continua dietro altre viste e programmi (`myapps/music`)
MARKDOWN	un editor di file Markdown (`.MD`, fino a 6K), come NOTEPAD. ^P mostra la pagina: titoli `#`, **grassetto**, *corsivo*, `codice`, elenchi, citazioni, linee, blocchi di codice e link sottolineati, senza i segni. Le frecce scorrono la pagina, RETURN o ^P tornano al testo. È sul suo dischetto, `markdown.dsk`, con un `README.MD`. Menu EDIT: COPY ^C, CUT ^X, PASTE ^V, SELECT ALL ^A; SHIFT e le frecce selezionano. Gli appunti (fino a 1K) sono comuni a tutti i programmi (`myapps/markdown`)
SYSINFO	mostra le versioni di V-OS e dell'API e lo spazio libero sui dischetti in A: e B: (^R di nuovo). Un esempio di `VOS_EXT` (`myapps/sysinfo`)
WEBGET	scarica file dal web con la scheda M4: GET A FILE (^G) chiede un indirizzo (HTTP, p. es. `example.com/index.html`) e un nome, e il file (fino a 16K) si salva sull'unità mostrata (OTHER DRIVE). Mostra anche lo stato della rete. Senza scheda M4: "NO M4 BOARD" (`myapps/webget`)
PONG	una palla e una racchetta con le frecce, punteggio e vite

CATCH	prendi le stelle che cadono
STARCAT	un gatto con costumi che cammina, una stella, una melodia da una lista
COUNTER	un contatore con i pulsanti +1, −1 e 0, in modo 1
FLAP	un uccello che batte le ali (costumi), e una stella che cambia costume ogni secondo
TICKER	conta i secondi e continua in background

- I programmi a blocchi sono file `.VBN`. Hanno un menu PROGRAM > STOP (CONTROL+Q).
- Quelli che hanno anche un `.VBS` (CALC, MINES, FLAP, TICKER) si aprono in Studio, così puoi guardarli o modificarli.
- `VBRUN.SYS` non è un programma: è il "player" di cui Studio ha bisogno.

4. Le lingue

V-OS parla sei lingue. L'inglese è integrato. Le altre cinque sono **pacchetti di lingua**, file sul dischetto che portano tre cose: tutti i testi di sistema, la tastiera e le lettere (il font) della lingua.

Lingua	File	Tastiera
English	integrato (`ENGLISH.MSG`)	quella inglese del CPC
Ελληνικά	`GREEK.LNG`	greca, con gli accenti
Français	`FRENCH.LNG`	AZERTY, cifre con SHIFT, tasti morti ^ e ¨
Español	`SPANISH.LNG`	Ñ sul tasto `:`, tasti morti ´ ¨ ` ^, e ¡ ¿
Deutsch	`GERMAN.LNG`	QWERTZ, Ü Ö Ä ß, tasti morti ´ e `
Italiano	`ITALIAN.LNG`	è é ò à ù ì su tasti propri

Cambiare lingua

- **In modo permanente:** nelle Impostazioni (F4), la riga LINGUA passa ciclicamente per tutte le lingue. Ognuna si carica subito da A:, e tutto lo schermo viene ridisegnato in quella lingua. Salvando (^S) resta anche per il prossimo avvio.
- **Rapidamente:** CONTROL+L (VISTA > CAMBIA LINGUA) passa dall'inglese alla lingua del pacchetto caricato e viceversa.
- **Solo la tastiera:** CONTROL+SHIFT passa la tastiera da quella inglese a quella del pacchetto e viceversa. La barra dei menu mostra a destra quale è attiva (EN, EΛ, FR, ...).

I simboli `@ # [ ] { } \ |` non sono sulle tastiere dei pacchetti, perché il CPC non ha AltGr. Scrivili con la tastiera inglese.

I testi dei programmi

I programmi hanno i propri testi in **due lingue**: l'inglese e, facoltativamente, un'altra (oggi il greco). Un programma appare nella sua seconda lingua solo quando il pacchetto è in quella lingua. Altrimenti appare in inglese. Quindi, con il francese, Studio appare in inglese.

Creare una nuova lingua

I sorgenti delle lingue sono in `lang/<code>/`:

- `messages.txt`: i testi di sistema. L'elenco di base è definito da `lang/en/messages.txt`, e quello che manca in un pacchetto appare in inglese.
- `keymap.txt`: i tasti diversi dalla tastiera inglese.
- `pack.txt`: il codice, il nome, il set di caratteri e il font.

`tools/mklng.py` li trasforma in un `.LNG`, e il Makefile lo mette sul dischetto. I pacchetti latini condividono il font `lang/latin1/font.txt`.

5. Studio: programmi con i blocchi

STUDIO è un editor a blocchi in stile Scratch che gira sul CPC stesso.

- Crei sprite e script con i blocchi.
- Il programma parte subito con CONTROL+R.
- Diventa un normale programma V-OS (`.VBN`) che si avvia anche dal Launcher.

Lo schermo

Studio usa uno schermo "diviso", con due zone:

- **In alto, in modo 0 (16 colori):**
  - **a sinistra, lo stage** (il palcoscenico) a metà dimensione
  - **a destra, il pannello SPRITES:** gli sprite con il loro colore e la loro posizione, la dimensione di quello selezionato (W, H), lo sfondo e il modo del programma (ad es. `STAGE 2 M1`), e il nome del file
- **In basso, in modo 1:**
  - **a sinistra, la palette** con i blocchi di una categoria (`< EVENTS >`)
  - **a destra, lo script**, una riga per blocco
  - **in fondo, la riga di stato**, con i tasti che funzionano dove ti trovi

TAB va dallo script alla palette, poi allo stage, e di nuovo allo script.

Menu

STUDIO		EDIT
NEW	^N	DELETE LINE
OPEN (`.VBS`)	^O	+ SPRITE (fino a 6)
SAVE	^S	− SPRITE
SAVE AS		BACKGROUND (il colore dello stage)
RUN	^R	SCREEN MODE (0 o 1)
IN BACKGROUND		
BACK	^Q	

- **RUN (^R):** salva `NAME.VBS`, scrive `NAME.VBN` e lo avvia. Il programma si ferma dal proprio menu (PROGRAM > STOP, ^Q), e poi Studio riparte con lo stesso programma aperto.
- **IN BACKGROUND:** chiede se il programma continua a girare quando è dietro un altro o dietro una vista, oppure si mette in pausa, che è il comportamento predefinito.
- **SCREEN MODE:**
  - **Modo 0:** 16 colori, 192 pixel di larghezza.
  - **Modo 1:** 4 colori (penne 0–3), 384 pixel di larghezza, e più testo.

La palette

←/→ cambiano la categoria, ↑/↓ scelgono un blocco, e RETURN lo aggiunge sotto la riga dello script.

Un clic su un blocco della palette fa lo stesso.

Un blocco che si apre (FOREVER, REPEAT, IF) entra insieme al suo END, e viene cancellato insieme a esso. Un blocco con un testo chiede subito il testo.

Lo script

Tasto	Cosa fa
↑/↓	riga
←/→	casella della riga (le caselle sono i valori dentro il blocco)
RETURN	senza casella selezionata: aggiunge il blocco della palette. Su una casella di testo: cambia il testo. Su una casella di tasto: aspetta che tu prema il tasto
CLR	cancella la riga
DEL	cancella l'ultima cifra di un numero
mouse	clic su una casella: la seleziona. Secondo clic: ne cambia il tipo

Il programma è un elenco di righe:

- Ogni `WHEN …` inizia uno script, per uno sprite o per lo stage. Lo script arriva fino al `WHEN` successivo.
- Il rientro viene da solo dai blocchi che si aprono e si chiudono.
- Le righe prima del primo `WHEN` vengono ignorate.
- Tutto quello che resta aperto viene chiuso alla fine dello script.

Cosa va in una casella

La riga di stato dice sempre che cosa accetta la casella.

Casella di valore:

Tasto	Valore
0–9, −	numero, un intero da −32768 a 32767
A–H	variabile. Di nuovo la stessa lettera: A1, A2 … A15, e di nuovo A
X, Y	la posizione dello sprite dello script
R e cifre	RANDOM: un numero da 1 a n
K e un tasto	KEY: 1 mentre è tenuto premuto, altrimenti 0
T e una cifra	TOUCHING: 1 quando lo sprite tocca lo sprite n
Z	AT EDGE: 1 quando lo sprite è al bordo dello stage
P	BUTTON: il numero dell'ultimo pulsante premuto
U	COSTUME: il numero del costume dello sprite (0: nessun costume)

Altri tipi di casella:

- **Variabile:** A–H, e di nuovo la stessa lettera per A1…A15.
- **Confronto:** `<`, `=`, `>`. SPAZIO dà "nessuno" (NONE), e allora IF controlla solo che il primo valore non sia 0.
- **Sprite** (nei blocchi `WHEN`): 0 lo stage, 1–6 uno sprite.
- **Tasto** (in `WHEN KEY`): RETURN e poi il tasto.
- **Testo:** RETURN, e scrivi fino a 14 caratteri.
- **Gruppo di variabili** (SAVE/LOAD VARS): A–H passa al gruppo di otto successivo (A-H, A1-H1 … A15-H15), e SPAZIO dà ALL.

Le lettere vengono lette come tasti, quindi funzionano anche con la tastiera greca.

Lo stage e gli sprite

Tasto	Cosa fa
frecce	muovono lo sprite selezionato
1–6	selezionano uno sprite
C	colore successivo
W, H	larghezza, altezza
K	quale costume indossa (nessuno, 1, 2, ...)
E	disegna il suo costume (uno nuovo, se non ne ha)
clic sullo stage	mette lì lo sprite

Uno sprite senza costume è un rettangolo colorato.

Costumi

E apre l'editor dei costumi al posto dello stage. Ha una griglia 16×16 con le 16 penne sotto.

Tasto	Cosa fa
frecce	casella
SPAZIO, RETURN	colora la casella
0–9, A–F	penna
.	trasparente
E, TAB	fatto
clic	su una casella: colora. Sulla riga delle penne: sceglie una penna

Un programma ha fino a 8 costumi, condivisi da tutti gli sprite. La dimensione di uno sprite con un costume è 16×16. Sullo stage, x viene arrotondata a un byte intero (2 pixel in modo 0, 4 in modo 1).

Tutti i blocchi

Nella colonna "Blocco", le caselle appaiono come nello script.

EVENTS

Blocco	Cosa fa
WHEN START *sprite*	lo script parte insieme al programma
WHEN KEY *tasto sprite*	parte quando si preme il tasto
WHEN CLICKED *sprite*	parte con un clic sullo sprite (o sullo stage)
WHEN I RECEIVE *n sprite*	parte quando viene inviato il messaggio n
SEND *n*	invia il messaggio n (n può essere anche una variabile)

> Un messaggio fa ripartire **dall'inizio** ogni script che lo aspetta, anche se era già in esecuzione. Per questo uno script non dovrebbe inviare un messaggio a se stesso.

CONTROL

Blocco	Cosa fa
FOREVER ... END	ripete per sempre
REPEAT *n* ... END	n volte
IF *a confronto b* ... [ELSE ...] END	esegue le righe solo se il confronto è vero

WAIT *n*	aspetta n frame (50 = un secondo)
WAIT UNTIL *a confronto b*	aspetta finché il confronto è vero
STOP	termina questo script

MOTION

Blocco	Cosa fa
GO TO *x y*	lo sprite va in x, y
CHANGE X *n*, CHANGE Y *n*	si sposta di n
SPEED *vx vy*	imposta la sua velocità
MOVE	un passo alla sua velocità
BOUNCE	al bordo dello stage, inverte la velocità

LOOKS/SOUND

Blocco	Cosa fa
COLOUR *n*	la penna del rettangolo
COSTUME *n*	indossa il costume n (1–8. 0 non cambia nulla)
NEXT COSTUME	il successivo, e dopo l'ultimo di nuovo il primo
SHOW, HIDE	
PRINT *valore* AT *x y*	stampa un numero
NOTE *n* FOR *frame*	suona una nota (numero MIDI: 60 = do, 69 = la 440 Hz) e aspetta che finisca

VARIABLES

Blocco	Cosa fa
SET *variabile* TO *valore*	
CHANGE *variabile* BY *valore*	addizione
DECREASE *variabile* BY *valore*	sottrazione
MULTIPLY *variabile* BY *valore*	
DIVIDE *variabile* BY *valore*	divisione intera
REMAINDER *variabile* BY *valore*	il resto della divisione
SAVE VARS *gruppo* TO *"nome"*	scrive le variabili in un file di testo (vedi sotto)
LOAD VARS *gruppo* FROM *"nome"*	le rilegge

WINDOWS

Blocco	Cosa fa
WINDOW *"titolo" larghezza altezza*	una finestra al centro dello stage, come i dialoghi di V-OS. Una nuova finestra prende il posto della precedente e dei suoi pulsanti
BUTTON *"testo"* AT *x y*	un pulsante di sistema. Si preme con un clic o con TAB e RETURN. I pulsanti sono numerati da 0, nell'ordine in cui appaiono
WHEN BUTTON *sprite*	parte quando viene premuto un pulsante qualsiasi. Il valore BUTTON (P) dice quale
TEXT *"testo"* AT *x y*	scrive un testo
LABEL *pulsante* TO *"testo"*	cambia il testo di un pulsante

Con una finestra aperta, TEXT, BUTTON e PRINT contano dall'angolo in alto a sinistra del suo

interno. La finestra si trascina per il titolo, insieme ai suoi pulsanti.

Le variabili

Ci sono 128 variabili, condivise da tutti gli script, chiamate A–H, A1–H1, …, A15–H15. All'inizio valgono tutte 0, e contengono interi da −32768 a 32767.

Salvare e ripristinare (SAVE VARS, LOAD VARS)

- `SAVE VARS ALL TO "GAME"` scrive tutte le 128 nel file `GAME.TXT`. Il `.TXT` viene aggiunto quando il nome non ha un tipo.
- Con un gruppo (`A-H`, `A1-H1`, …) scrive solo quelle 8.
- Il file è di testo, una riga per variabile:

```
A=12 B=0 A1=-5
```

- `LOAD VARS ALL FROM "GAME"` cambia le variabili che il file nomina. Con un gruppo, cambia solo quelle che sono anche nel gruppo. Le righe che non capisce vengono ignorate, e se il file manca non cambia nulla.
- Così un file può ripristinare tutte le variabili o solo un gruppo di otto, ad es. le posizioni salvate di un gioco.

I file

- **`NAME.VBS`:** il programma come lo modifichi. OPEN lo apre, e vengono letti fino a 5K.
- **`NAME.VBN`:** il programma pronto da eseguire. RUN lo scrive, e si avvia anche dal Launcher. È il player (`VBRUN.SYS`) insieme al programma. Il player compila i blocchi in codice Z80 all'avvio, quindi i programmi sono veloci.

Un `.VBS` si può creare anche da un testo sul PC, con `tools/mkvbs.py`. La sintassi è descritta all'inizio del file, e gli esempi sono `blocks/examples/*.vbt` (CALC, MINES, FLAP, TICKER, VARS).

Limiti

Righe (blocchi)	240
Sprite	6
Script (`WHEN …`)	16
Variabili	128
REPEAT in un programma	64 (oltre, condividono un contatore)
Blocchi aperti annidati uno dentro l'altro	16
Testi	32, fino a 14 caratteri ciascuno
Costumi	8, 16×16
Pulsanti	64
Codice	almeno 4.000 byte, anche per il programma più grande. Quello che non ci sta viene ignorato

Consigli

- **Script dello stage:** i blocchi di movimento e di aspetto non fanno nulla, e X, Y, TOUCHING e AT EDGE danno 0.

- **TOUCHING:** vale solo quando entrambi gli sprite sono visibili.
- **Tempi:** gli script girano "insieme". Ogni script cede il turno alla fine di ogni ciclo e a ogni WAIT. Per far aspettare uno script un altro, usa una variabile e `WAIT UNTIL` (vedi MINES).
- **Esempi pronti:** CALC, MINES, FLAP e TICKER si aprono in Studio (OPEN).

6. I blocchi sul PC: l'editor web e `vbc.py`

Oltre a Studio, che gira sul CPC, c'è anche un editor a blocchi per il PC. Il linguaggio è lo stesso, con qualche blocco in più: liste, rumore, dimensione degli sprite, testo colorato.

L'editor

Apri `blocks/editor.html` in un browser. È basato su Blockly ed è disponibile in greco o in inglese.

- **Categorie:** eventi, controllo, movimento, aspetto, testo, operatori, sensori, variabili, liste e finestre.
- **Sprite:** rettangoli o costumi. Si definiscono in una tabella, e lo stage li mostra con i colori del CPC. Si possono trascinare con il mouse.
- **File:** apre e salva file `.vbp`. Sono JSON con tutto il programma, quindi si possono anche scrivere a mano.

Compilazione

```
python3 blocks/vbc.py pong.vbp --dsk build/vos.dsk
```

`vbc.py` scrive `build/NAME.ASM`, con un commento per ogni blocco. Lo assembla con RASM in `NAME.VBN`, con una tabella di rilocazione perché giri accanto ad altri programmi, e con `--dsk` lo mette sul dischetto.

Opzione	Cosa fa
`-o build`	cartella di uscita
`--asm-only`	solo il `.ASM`
`--dsk build/vos.dsk`	anche sul dischetto

Il formato `.vbp`

La descrizione completa è all'inizio di `blocks/vbc.py`: eventi, comandi, valori, costumi, icona. In breve:

```
{"name": "PONG", "mode": 0, "background": 0,
 "sprites": [{"name": "ball", "x": 90, "y": 100, "w": 6, "h": 6,
              "colour": 8, "vx": 2, "vy": 2, "visible": true}],
 "variables": {"score": 0},
 "scripts": [{"sprite": "ball", "when": {"event": "start"},
              "do": [{"block": "forever", "do": [{"block": "move"}, {"block": "bounce"}
```

- **Variabili e liste:** hanno nomi, non lettere. Le liste hanno una lunghezza massima fissa.
- **Icona:** `"icon"` dà un'icona per il Launcher.
- **Un programma con molte parti simili:** può scriverlo uno script Python. Vedi `tools/mkbricks.py`, che scrive `bricks.vbp` con 32 mattoni.

Esempi

In `blocks/examples/`:

File	Programma
`pong.vbp`	PONG

`catch.vbp`	CATCH
`starcat.vbp`	STARCAT: costumi, una melodia da una lista
`counter.vbp`	COUNTER: una finestra e pulsanti
`bricks.vbp`	BRICKS (scritto da `tools/mkbricks.py`)

I file `*.vbt` sono programmi di Studio in forma di testo, per `tools/mkvbs.py`. Vedi il capitolo 5.

Studio o editor web?

	Studio	**editor web**
Dove	sul CPC	sul PC
File	`.VBS`	`.vbp`
Variabili	128, A–H15	con nomi
Liste, rumore, dimensione degli sprite	no	sì
SAVE/LOAD VARS	sì	no
Il programma	ha bisogno di tutta la memoria	piccolo, gira accanto ad altri

7. L'SDK di V-OS: programmi in assembly

Con questo SDK scrivi i tuoi programmi in assembly Z80. Girano dentro V-OS come DEMO o CLOCK: compaiono nel Launcher, nella Scrivania e nel Commander, e hanno il proprio menu, pulsanti, dialoghi, suono e file. Girano anche accanto ad altri programmi.

Avvio rapido

```
make new-app NAME=hello
```

Questo crea la cartella `myapps/hello/` da `sdk/template/`. Contiene tre file:

- `hello.asm`: il programma. Le frecce muovono un riquadro, c'è un pulsante BEEP, un contatore dei secondi e un menu proprio (ABOUT, BACK) accanto a VISTA e AIUTO.
- `texts.txt`: i suoi testi, in inglese e in greco.
- `icon.txt`: la sua icona, 16×16, per il Launcher.

```
make app NAME=hello
```

Questo costruisce `build/HELLO.BIN` e lo mette sul dischetto `build/vos.dsk`.

```
make run
```

Questo apre il dischetto in Caprice32. HELLO è nel Launcher, nell'ultima pagina.

- Il semplice `make` costruisce tutti i programmi in `myapps/` e li mette sul dischetto, insieme a V-OS.
- Il nome ha 1–8 lettere minuscole o cifre e inizia con una lettera. Non può essere il nome di un programma di V-OS (ad es. demo).
- Servono RASM, Python 3 e iDSK, come per tutto V-OS.
- In Caprice32 il programma lo avvii tu, dal Launcher.

Un programma più grande:

- Metti altri file `.asm` nella sua cartella (ad es. `art.asm` con gli sprite) e includili con `include "art.asm"`. Il build li trova, e ricostruisce il programma quando cambiano.
- Quello che metti nella sottocartella `disc/` (ad es. `disc/LEVEL1.DAT`) va così com'è sul dischetto, per `VOS_LOAD`.
- Con una sottocartella `owndisc/`, il programma ha anche un dischetto suo, `build/<nome>.dsk`, con il `.BIN` e i file di `owndisc/`. Non va su `vos.dsk`. Lo metti in B: e lo avvii dal Launcher (TAB, l'altra unità). Il drive da cui è partito, per caricare da lì i suoi file, è in `VOS_SELF`: `db drive`, poi il suo nome (11 byte).
- **Un file da aprire (versione 14):** quando RETURN nella Scrivania o nel Commander apre un file nel tuo programma, `VOS_OPEN` (`#B000`) contiene `db 1, unità, nome di 11 byte` (0: nessuno). Leggilo in `init`, con A = 0 e anche con A = 2 (il programma era già in esecuzione), poi scrivici 0. I tipi e i loro programmi sono nella tabella `ft_open` del kernel (`src/kernel/files_ext.asm`).
- Un programma con tabella di rilocazione (come quelli dell'SDK) può arrivare a 12K. Per il resto, carica file nel banco dati.

Come funziona un programma

Un programma è un `.BIN`, costruito per `VOS_APP_ADDR` (`#4000`). Inizia così:

```
        include "api/vos_api.inc"
        org VOS_APP_ADDR
        jp init                 ; +0
        jp event                ; +3  HL -> l'evento
        jp redraw               ; +6
        db "VA", VOS_API_VERSION
        include "app_icon.asm"  ; +12 l'icona (facoltativa)
```

- **`init`** imposta modo, palette, menu e pulsanti, attiva `VE_FRAME` se serve, e disegna tutto.
  - A = 0: un avvio da zero.
  - A = 2: il programma torna da una vista o da un altro programma. Il suo stato è come l'aveva lasciato; imposta di nuovo quanto sopra e disegna.
- **`event`** viene chiamata per ogni evento e ritorna subito. Un programma non aspetta mai da solo (nessun ciclo di attesa). Per movimento o tempo, `VOS_TICKS` con A = 1, e poi arriva un `VE_FRAME` 50 volte al secondo.
- **`redraw`** ridisegna tutto, ad es. quando cambia la lingua o si chiude l'accessorio di scrivania.
- **Uscita:** `VOS_EXIT`, che torna alla vista che l'ha avviato.

L'ordine dentro `event`

```
event:  ld a,(hl)
        cp VE_FRAME : jp z,frame        ; i frame
        push hl
        call VOS_UI_EVENT               ; menu, pulsanti, CONTROL+lettera
        pop hl
        or a : jr nz,command            ; A = il comando
        call VOS_MENU_OPEN : ret nz     ; menu aperto: i tasti sono suoi
        ...                             ; i tuoi tasti e clic
command: ...
        jp VOS_COMMAND                  ; quello che non è tuo (VISTA, AIUTO)
```

Mentre un menu è aperto, non disegnare. Altrimenti disegnerai sopra di esso. Su `VE_FRAME`, aggiorna lo stato, ma disegna solo se `VOS_MENU_OPEN` dà Z. I dialoghi (`VOS_DIALOG`, `VOS_INPUT`, `VOS_FILE_PICK`) hanno un proprio ciclo, quindi mentre sono aperti non arriva nessun evento.

Gli eventi

`HL` → 8 byte: `db type, code, char, mods; dw x, y`.

type	cosa	code / char / x, y
`VE_KEY_DOWN` 1, `VE_KEY_REPEAT` 3, `VE_KEY_UP` 2	tasto	code = numero del tasto (`VK_*`, riga della tastiera·8 + bit), char = il carattere nella lingua del pacchetto
`VE_PTR_DOWN` 5, `VE_PTR_UP` 6, `VE_PTR_MOVE` 4	puntatore	x, y in pixel del modo
`VE_FRAME` 8	frame (con `VOS_TICKS`)	—
`VE_LANG` 7	la lingua della tastiera è cambiata	—

Tre tipi di programma

- **Con una tabella di rilocazione (il predefinito dell'SDK):** `make app` lo costruisce due volte, a `#4000` e a `#4100`. `tools/mkreloc.py` aggiunge la tabella dalla differenza.
  - Gira accanto ad altri programmi, nei 12K da `#5000`, e riprende da dove si era fermato (F6 mostra i programmi in esecuzione).
  - I suoi dati devono stare dentro i suoi stessi byte (etichette, `ds`), mai a indirizzi fissi.
  - Se `mkreloc.py` dà un errore, da qualche parte un indirizzo è usato in un modo che non sopravvive alla rilocazione. Ad es. `ld a,label & 255`, o solo il byte basso di un indirizzo.
- **Senza tabella:** prende tutto il banco (fino a 16K, codice e dati), ma gira solo da solo, a meno che non ci sia un'espansione di memoria. Serve una tua regola nel Makefile (vedi `STUDIO.BIN`).
- **Accessorio di scrivania:** fino a 4K, con una tabella. Si apre con F5 sopra qualsiasi cosa, e `init` riceve A = 1. Vedi `apps/clock/clock.asm` e la descrizione in `src/api/vos_api.inc`.

Regole

- **Registri:** le chiamate alterano tutti i registri tranne IX. Gli argomenti vanno in A, BC, DE, HL (IX per `VOS_RECT` e `VOS_BLIT`). I risultati tornano in A o HL. In caso di errore, il carry è impostato e A = numero dell'errore, che `VOS_ERROR` mostra.
- **Memoria:** non mappare un altro banco a `#4000`, perché il programma ti sparirebbe da sotto. Per altri 16K c'è il banco dati (`VOS_BANK_GET`, `VOS_BANK_PUT`, `VOS_DATA`). È condiviso con le viste e con altri programmi, e conserva i suoi dati solo mentre il programma è in esecuzione.
- **Coordinate:** x in pixel del modo (modo 0: 0–191, modo 1: 0–383), y da 0 a 258. La barra dei menu occupa y 0–11.
- **RASM:** le etichette non distinguono maiuscole e minuscole (`BOX` e `box` sono la stessa). L'esadecimale si scrive con `#` e non con `&`.

I formati dei dati

- **Barra dei menu** (`VOS_SET_BAR`): `db count; dw menu, ...`. I comuni VISTA e AIUTO vengono da `VOS_MENU_VIEW` e `VOS_MENU_HELP`.
- **Menu:** `db title, count` e per ogni voce `db text, command, letter, 0`.
  - text: `VOS_SEPARATOR` per una linea
  - command: 1.., quello che restituisce `VOS_UI_EVENT`
  - letter: scorciatoia CONTROL+lettera, o 0
- **Pulsanti** (`VOS_SET_WIDGETS`, poi `VOS_DRAW_WIDGETS`): `db count`, e per ogni pulsante `dw x, y; db larghezza in caratteri, text, command, 0`.
- **Dialogo** (`VOS_DIALOG` → A = pulsante, 0 il primo, ESC dà l'ultimo): `db title, text, numero di pulsanti (1-3), pulsante, ...`. Per i pulsanti, `VM_OK`, `VM_YES` e `VM_NO`.
- **Testi** (`VOS_TEXTS`): creati da `tools/mktexts.py` a partire da `texts.txt`, con righe `NAME | English | Greek`.
  - Dà `T_NAME` (`VT_FIRST` = `#C0` e oltre) e la tabella `texts:`. Fino a 64 testi.
  - Un `|` senza spazi intorno inizia una nuova riga nei dialoghi.

    - I testi vanno dove viene chiesto un "testo" (menu, pulsanti, dialoghi). Con `VOS_MSG` ottieni il loro indirizzo per `VOS_PRINT`.

- **Palette** (`VOS_PALETTE`): 17 colori del firmware (0–26): 16 penne e il bordo. In modo 1 contano le penne 0–3.
- **Rettangolo** (`VOS_RECT`, IX →): `dw x, y, w, h; db pen`.
- **Icona:** 16 righe di 16 caratteri in `icon.txt`. Ogni carattere è una penna in esadecimale (modo 0, palette del Launcher) o `.` per trasparente.
- **Nomi di file** (`VOS_LOAD`, `VOS_SAVE`, `VOS_DELETE`, `VOS_RUN`): 11 byte, `"NAME EXT"`. Unità 0 = A:, 1 = B:.

L'API (versione 17)

Chiamata	Ingresso → uscita	Cosa fa
`VOS_EXIT`	—	fine, ritorno alla vista che l'ha avviato
`VOS_TICKS`	A = 1 / 0 (+ `VOS_PTR_KEYS`)	`VE_FRAME` 50 volte al secondo, o nessuno. Con `VOS_PTR_KEYS` (2) le frecce e COPY muovono anche il puntatore e fanno clic
`VOS_FRAMES`	→ HL (e A bit 16–23)	frame dall'avvio
`VOS_RANDOM`	→ A	0–255
`VOS_COMMAND`	A = comando	i comandi comuni di VISTA e AIUTO (6–19). Numera i tuoi comandi da `VC_APP` (32) in su, altrimenti una voce di VISTA o AIUTO esegue uno dei tuoi
`VOS_ERROR`	A = errore	dialogo di errore
`VOS_MODE`	A = 0 / 1	modo 0 (192×259, 16 penne) o 1 (384×259, 4 penne)
`VOS_PALETTE`	HL → 17 colori	
`VOS_CLS`	A = penna	cancella lo schermo
`VOS_PLOT`	HL = x, DE = y, A = penna	un pixel
`VOS_RECT`	IX → rettangolo	rettangolo pieno
`VOS_COLOURS`	A = penna del testo (+ `VOS_BOLD`, `VOS_ITALIC`, `VOS_UNDERLINE`), C = sfondo	per `VOS_PRINT` e `VOS_CHAR`. Gli stili nascono dallo stesso font, per ogni lingua
`VOS_PRINT`	HL = x, DE = y, BC → testo, 0	
`VOS_CHAR`	HL = x, DE = y, A = carattere	
`VOS_MSG`	A = testo → HL	l'indirizzo di un testo (`T_*` o uno di sistema)
`VOS_SET_BAR`, `VOS_DRAW_BAR`	HL → barra dei menu	
`VOS_UI_EVENT`	(l'evento corrente) → A	comando di menu, pulsante o scorciatoia, 0 per nessuno
`VOS_DIALOG`	HL → dialogo → A	
`VOS_INPUT`	A = titolo, HL → testo, C = massimo → carry se annullato	inserimento di testo (servono C + 2 byte)
`VOS_SET_WIDGETS`, `VOS_DRAW_WIDGETS`	HL → pulsanti	

`VOS_MENU_VIEW`, `VOS_MENU_HELP`	→ HL	i menu comuni
`VOS_CATALOG`	A = unità → A = file, HL → elenco	16 byte ciascuno: 11 il nome, a +12 la dimensione in K. Con A = unità + `VOS_CAT_EXT` e HL → `"EXT"`, solo i file con quell'estensione
`VOS_FILE_PICK`	A = titolo → A = file, carry se annullato	dialogo di scelta, dopo `VOS_CATALOG`
`VOS_LOAD`	A = unità, HL → nome, DE = dove → BC = lunghezza	
`VOS_SAVE`	A = unità, HL → nome, DE = da, BC = lunghezza	file binario, che sostituisce il vecchio
`VOS_DELETE`	A = unità, HL → nome	
`VOS_TEXTS`	HL → tabella dei testi	i testi del programma
`VOS_MENU_OPEN`	→ NZ se un menu è aperto	
`VOS_KEY_HELD`	A = tasto → NZ mentre è premuto	per i giochi
`VOS_BLIT`	IX → `dw x, y; db bytes, lines; dw source; db source stride`	immagine: coppie (maschera, pixel)
`VOS_SOUND`	A = canale 0–2, HL = periodo (bit 15: rumore), C = volume 0–15, B = frame	una nota; 440 Hz è il periodo 142
`VOS_SPLIT`	HL → `dw y1, y2; db 8 colors` (0 = disattiva)	schermo misto: modo 0 tra y1 e y2
`VOS_RUN`	A = unità, HL → nome	avvia un altro programma, e poi questo riparte
`VOS_KEEP`	→ HL → 16 byte	memoria che resta tra un programma e l'altro
`VOS_BANK_GET`, `VOS_BANK_PUT`	HL = da, DE = a, BC = byte	copia da e verso il banco dati
`VOS_BACKGROUND`	A = 1 / 0	continua a girare o si blocca quando è dietro un altro
`VOS_DRAG`	HL = x, DE = y, BC = w, A = h → carry: nuovi HL, DE	trascinamento di una finestra per il titolo
`VOS_FIT`	come sopra → HL, DE	riporta una finestra dentro lo schermo
`VOS_EXT`	A = una chiamata `VX_*`, BC, DE, HL come vuole → A, HL, C, DE, carry	le chiamate dopo la tabella (versione 15): `VX_VERSION` → A = API, HL = versione di V-OS (BCD). `VX_FREE` C = unità → A = K liberi. Sconosciuta: carry `VX_CLIP_PUT` HL = testo, BC = byte → gli appunti (fino a 1K). `VX_CLIP_GET` DE = dove, BC = spazio → HL = dimensione (versione 16). `VX_M4` HL → un comando per la scheda M4 (db dimensione, dw comando, dati), DE → la risposta, BC = byte; carry: niente scheda M4 (versione 17).

I dettagli completi, ad es. sul background, sull'accessorio e su `VOS_SPLIT`, sono in `src/api/vos_api.inc`. Per esempi, vedi `apps/demo/demo.asm` (modo 0, movimento), `apps/clock/clock.asm` (accessorio) e `apps/mixed/mixed.asm` (schermo misto). Un gioco intero con sprite, un livello letto

da file e suono è `myapps/vrouves/`.

Un assistente IA che scrive un programma deve leggere `AI.md` (in inglese): le stesse regole, più brevi, con le trappole di RASM e del banco dati.

Test

`make test-sdk` costruisce il modello come TEMPLATE.BIN e lo apre dal Launcher in Caprice32, con pressioni di tasti definite da `src/kernel/test_script.asm` (script 15). Cattura schermate in `build/shots`: la sua schermata, il riquadro dopo che si è mosso, il suo menu, il dialogo ABOUT e il ritorno al Launcher.

Per un tuo test automatico, copia lo script 15 e il target `test-sdk` del Makefile.

8. Sviluppare V-OS

Per chi lavora su V-OS stesso. I programmi *per* V-OS sono descritti nell'SDK.

Cosa serve

- RASM, l'assembler
- iDSK, per le immagini disco
- Python 3, per gli strumenti. `make docs` ha bisogno anche di `markdown` e `pymupdf`.
- GNU make 4.3 o successivo
- per i test, Caprice32 (snap `caprice32`)

Build

Comando	Cosa fa
`make`	`build/vos.dsk`: il kernel, le viste, le lingue, tutti i programmi e `myapps/*`
`make run`	apre il dischetto in Caprice32 (`RUN"VOS"`)
`make shot`	si avvia senza finestra e cattura una schermata
`make docs`	il manuale in PDF: `docs/V-OS.pdf` (greco) e `docs/V-OS-en.pdf`, `-fr`, `-es`, `-de`, `-it`
`make windows`	`build/V-OS-windows.zip`: `V-OS.exe` per Windows, con CPCEMU 3.0.2 (CPC 6128, 576K, mouse AMX, scheda M4), `vos.dsk` in A: e `vrouves.dsk` in B:. V-OS parte da solo. Servono WSL e lo zip portatile di CPCEMU (`CPCEMU_ZIP=…`)
`make linux`	`build/V-OS-linux.tar.gz`: lo script `V-OS` per Linux (x86_64), con CPCEMU 2.5 (la 3.0.2 non è uscita per Linux) e le stesse impostazioni e dischetti. Servono SDL2 sul sistema e il tar.gz di CPCEMU per Linux (`CPCEMU_TGZ=…`)
`make songs`	i brani di Arkos Tracker 3 come file `.MUS` in `myapps/music/owndisc` (`tools/mkmusic.py`; serve Arkos Tracker 3, `ARKOS=…`)
`make new-app NAME=x`, `make app NAME=x`	il tuo programma (SDK)
`make clean`	cancella `build/`

Struttura del repository

Cartella	Contenuto
`src/vos.asm`	la radice della build: loader e kernel, scrittura sul dischetto
`src/boot/`	il loader che AMSDOS esegue a `#4000`, rilevamento dell'espansione RAM
`src/kernel/`	vettori, interrupt, file system, viste, API (`api.asm`), impostazioni, estensione del kernel (`ext.asm`), script di test (`test_script.asm`)
`src/drivers/`	Gate Array, CRTC, tastiera, mouse AMX, FDC, AY
`src/gfx/`	grafica overscan, testo, schermo misto
`src/ui/`	menu, pulsanti, dialoghi, inserimento di testo, selettore di file
`src/shell/`	le viste come overlay: Scrivania, Launcher, Commander, Impostazioni
`src/api/` `vos_api.inc`	l'API per i programmi
`lang/`	le lingue (testi, tastiere, font)

`apps/`	i programmi in assembly: DEMO, MIXED, STUDIO, CLOCK, e le loro icone
`blocks/`	l'editor web, `vbc.py`, `runtime.asm`, il player di Studio (`player.asm`), esempi
`sdk/`	l'SDK: guida e modello di programma
`tools/`	strumenti: font, lingue, testi, icone, rilocazione, `.VBS` da testo, schermate, PDF
`docs/`	questo manuale in greco (`manual/`) e in inglese, francese, spagnolo, tedesco e italiano (`manual/en/`, `fr/`, `es/`, `de/`, `it/`), i PDF, i requisiti (`REQUIREMENTS.md`)

La memoria, in breve

Area	Cosa contiene
`#0000-#3DFF`	il kernel. Nella pagina zero, dopo i vettori di restart, la tabella dell'API a `#0108`
fino a `#4000`	lo stack del kernel
`#4000-#7FFF`	la "finestra" dei banchi
`#8000-#FFFF`	lo schermo overscan da 32K. Nei suoi spazi vuoti, buffer del kernel

I banchi che entrano nella finestra:

Banco	Cosa contiene
4	buffer dei file e di caricamento, e il banco dati del programma in esecuzione
5	la vista a `#4000-#4FFF`, e da `#5000` i programmi con una tabella di rilocazione (o un programma senza tabella in tutto il banco)
6	quello che coprono i menu e i dialoghi, e da `#7000` l'accessorio di scrivania
7	il pacchetto di lingua, gli elenchi del Commander, i testi inglesi, e da `#7000` l'estensione del kernel

Con un'espansione di memoria, i suoi banchi ospitano programmi. I dettagli, indirizzo per indirizzo, sono in `docs/REQUIREMENTS.md` (§4.2 e in ogni sezione "Υλοποίηση M..." (implementazione)).

Convenzioni

- **RASM:** l'esadecimale si scrive `#C000` e non `&C000`, perché RASM legge `&` come AND. Le etichette non distinguono maiuscole e minuscole.
- **Commenti:** in inglese, brevi, come nel codice esistente. Ogni file inizia con un blocco che dice che cosa fa.
- **Documentazione:** in greco. `MILESTONES.md` (inglese), `MILESTONES.el.md` (greco) e `docs/REQUIREMENTS.md` hanno una sezione per ogni milestone. Il manuale è scritto in greco e tradotto nelle altre cinque lingue di V-OS.
- **Testi:** i testi di sistema sono in `lang/*/messages.txt`, e i testi dei programmi in `texts.txt` (per `tools/mktexts.py`).

Test

I test eseguono V-OS in Caprice32 senza finestra e catturano schermate in `build/shots/<delay>/`.

Come funzionano:

- Caprice32 senza finestra non può premere frecce, COPY o tasti f. Per questo le pressioni dei tasti sono fatte da una build speciale: un `-DTEST_SCRIPT=n` inserisce lo script n in `src/kernel/test_script.asm`, una tabella di tasti per frame.
- Ogni target `make test-…` costruisce un dischetto con il suo script e cattura una schermata a ogni ritardo in `…_SHOTS`.

- Il ritardo si misura in pressioni di "Z": circa 2,5–3,5 frame ciascuna. La corrispondenza non è fissa, quindi un momento si cattura con due o tre ritardi intorno a esso.
- Quando V-OS scrive sul dischetto, Caprice32 non si chiude da solo, e il test finisce per timeout. Questo è previsto in test-vars, test-costume e altri.
- Le schermate le guardi tu, ad es. combinate in un'unica immagine con `ffmpeg … hstack/vstack`.

Target	Cosa testa
`test-input`	tasti, Studio: blocchi, file, esecuzione
`test-calc`	CALC con clic, poi `CALC.VBS` in Studio
`test-costume`	costume in Studio, esecuzione, riapertura
`test-acc`	l'orologio come accessorio di scrivania
`test-multi`, `test-switch`, `test-expand`, `test-behind`	più programmi, F6, espansione di memoria, background
`test-drag`	trascinamento delle finestre
`test-langs`, `test-lang`	le lingue, il salvataggio della lingua
`test-mines`, `test-bricks`	i giochi
`test-vars`	le 128 variabili, SAVE/LOAD VARS
`test-sdk`	il modello dell'SDK
`test-vrouves`	VROUVES dal suo dischetto in B:, tramite il Launcher (TAB): costruzione, l'ondata 1, i soldi, HIGH SCORES
`test-notepad`	NOTEPAD: scrittura, DEL, il dialogo di salvataggio, OPEN con solo file `.TXT`
`test-paint`	PAINT: linea, rettangolo, riempimento, i menu, poi i livelli (nasconderne uno)
`test-about`	AIUTO > INFORMAZIONI SU V-OS
`test-music`	MUSIC da `music.dsk` in B:: un brano suona e continua dietro il Launcher
`test-markdown`	MARKDOWN da `markdown.dsk` in B:: apre `README.MD`, la pagina con ^P, una schermata più giù, ritorno al testo
`test-open`	apertura per tipo nel Commander: `README.MD` in MARKDOWN da B:, un `.TXT` in NOTEPAD da A:
`test-sysinfo`	SYSINFO dal Commander: versione, API, K liberi su A: e B:, ^R, ^Q
`test-clip`	gli appunti: selezione con SHIFT, ^C e ^V in NOTEPAD, ^A, ^X, ^V, poi ^V in MARKDOWN
`test-webget`	a mano, nel CPCEMU di `make linux` (scheda M4; servono uno schermo e internet): WEBGET scarica `example.com/index.html`. Chiudi CPCEMU con F12 quando appare SAVED, e si cerca `INDEX.HTM` sul dischetto
`test-amsdos`, `test-reboot`, `test-system`	file di V-OS in BASIC, riavvio, dischetto SYSTEM

Nuovo script:

1. Aggiungi un `IF TEST_SCRIPT == n` all'inizio della catena in `test_script.asm`, con `ELSE` prima del successivo e un `ENDIF` in più alla fine.
2. Aggiungi un target al Makefile, come `test-sdk`.

Ogni tasto si scrive come `dw from, to : db row, bit` della tastiera. Il numero del tasto è riga·8 + bit, ad es. RETURN = 18 = riga 2, bit 2.

Senza emulatore: il codice di un programma gira anche su uno Z80 dentro Python (il pacchetto

`z80`). Carichi il `.BIN` a `#4000` e fai sì che le chiamate dell'API ritornino subito (o facciano quello che serve, ad es. file in un dizionario). Poi chiami init ed event e leggi la memoria. È molto più veloce delle schermate per gli errori in numeri, file e salti. Gli indirizzi delle etichette vengono da `rasm … -s -sl` (un file `.sym`).

A ogni milestone

1. Codice, e un test in Caprice32: nuovo o esistente, con schermate.
2. I test che la modifica tocca.
3. Una sezione in `MILESTONES.md` e `MILESTONES.el.md`, e una sezione "Υλοποίηση Μ…" (implementazione) in `docs/REQUIREMENTS.md`.
4. **Aggiorna il manuale in tutte e sei le lingue** (`docs/manual/` in greco, `docs/manual/<code>/` per en, fr, es, de, it, e `sdk/README.md`, `sdk/README.<code>.md` se l'API cambia), poi `make docs` per tutti i PDF.
5. Commit e push.